

Dart Language --- Full Notes

Based on Dart 3.13 (stable, August 2026)

Table of Contents

1. Introduction to Dart
2. Setup & Tooling
3. Variables & Data Types
4. Operators
5. Control Flow
6. Functions
7. Null Safety
8. Collections
9. Object-Oriented Programming
10. Modern Dart 3.x Features (Records, Patterns, Sealed Classes)
11. Primary Constructors (Dart 3.13, new)
12. Asynchronous Programming
13. Error Handling
14. Packages, pubspec.yaml & Compilation Targets
15. Practice Exercises

1. Introduction to Dart

Dart is a statically-typed, object-oriented, client-optimized language developed by Google. It powers Flutter (mobile/web/desktop UI) but also runs standalone on servers, CLI tools, and compiles to native machine code, JavaScript, or WebAssembly.

Key properties:

- **Sound null safety** by default since Dart 2.12 -- no more null-pointer style crashes.
- **Everything is an object**, including numbers and functions.
- Supports both **JIT** (fast dev iteration, hot reload) and **AOT** (fast startup, native performance) compilation.
- Can compile to **JavaScript and WebAssembly** for web deployment.
- First-class support for **async/await** and lightweight concurrency via **isolates** (no shared-memory threads).

2. Setup & Tooling

- Install the Dart SDK (bundled with Flutter, or standalone via dart-sdk).
- Run a file: `dart run main.dart`
- Format code: `dart format .`
- Analyze/lint: `dart analyze`
- Create a new project: `dart create my_app`
- Try snippets instantly in **DartPad** (no install needed) -- great for quick experiments.
- Recommended editor: VS Code with the Dart/Flutter extensions.

3. Variables & Data Types

```
var name = 'Ali';           // type inferred as String
String city = 'Karachi';    // explicit type
final age = 21;             // can be set once at runtime
const pi = 3.14159;         // compile-time constant

int a = 10;
double b = 10.5;
num c = 10;                // int or double
bool isActive = true;
String greeting = "Hello";
```

var vs final vs const:

- `var` -- mutable, type inferred once assigned.
- `final` -- assigned once, but the value can be computed at runtime (e.g. `DateTime.now()`).
- `const` -- compile-time constant, value must be known before the program runs.

Type conversion:

```
int x = int.parse('42');
double y = double.parse('3.14');
String s = 42.toString();
```

Strings:

```
String name = 'Ali';
String greeting = "Hi, $name!";           // interpolation
String multi = "Multi-line\nstring\nusing escapes";
String raw = r'C:\Users\ali';             // raw string, no escapes processed
```

4. Operators

Category	Operators
Arithmetic	+ - * / % ~/ (~ / = integer division)
Relational	== != > < >= <=
Logical	&& !
Assignment	= += -= *= /= ??=
Null-aware	?? ??= ?. ?..
Cascade	.. (chain calls on same object)
Type test	is, is!, as (cast)
Spread	..., ...? (null-aware spread)

Cascade example:

```
var sb = StringBuffer()
  ..write('Hello')
  ..write(' ')
  ..write('World');
```

5. Control Flow

```
if (age >= 18) {
  print('Adult');
} else if (age > 12) {
  print('Teen');
} else {
  print('Child');
}

for (var i = 0; i < 5; i++) { print(i); }
for (var item in [1, 2, 3]) { print(item); }

while (age < 30) { age++; }
do { age--; } while (age > 0);

switch (day) {
  case 'Mon':
    print('Start of week');
    break;
  default:
    print('Other day');
}
```

Modern switch expression (Dart 3+):

```
String result = switch (day) {
  'Mon' || 'Tue' || 'Wed' || 'Thu' || 'Fri' => 'Weekday',
  'Sat' || 'Sun' => 'Weekend',
  _ => 'Unknown',
};
```

6. Functions

```
int add(int a, int b) {
  return a + b;
}

int square(int x) => x * x;    // arrow syntax

void greet(String name, {String greeting = 'Hello'}) {
  print('$greeting, $name!');
}
greet('Ali', greeting: 'Hi');  // named parameter

void log(String msg, [String? tag]) {
  print(tag != null ? '[$tag] $msg' : msg);
}
log('starting', 'DEBUG');      // positional optional parameter

// Functions as first-class objects
void process(int x, int Function(int) transform) {
  print(transform(x));
}
process(5, (x) => x * 2);
```

Required named parameters:

```
void createUser({required String name, required int age}) { }
```

7. Null Safety

Dart enforces **sound null safety**: all types are non-nullable by default.

```
String name = 'Ali';           // cannot be null
String? nickname;              // can be null, defaults to null

nickname ??= 'Unknown';        // assign only if currently null
String display = nickname ?? 'Guest';  // fallback value

int? length = nickname?.length;  // safe navigation, returns null if nickname is null

String forced = nickname!;      // null assertion -- throws if actually null, use carefully
```

late keyword -- for non-nullable variables initialized after declaration:

```
late String description;
void init() { description = 'Loaded'; }
```

Best practices:

- Prefer non-nullable types; only use ? when a value genuinely may be absent.
- Avoid ! unless you are certain the value isn't null -- it throws at runtime otherwise.
- Use late for lazy initialization, not to silence the compiler.

8. Collections

```
// List
List<int> nums = [1, 2, 3];
nums.add(4);
var doubled = nums.map((n) => n * 2).toList();
var evens = nums.where((n) => n.isEven).toList();

// Set -- unique values, unordered
Set<String> tags = {'ai', 'ml', 'dart'};

// Map -- key-value pairs
Map<String, int> scores = {'Ali': 90, 'Sara': 85};
scores['Ali'];
scores.forEach((key, value) => print('$key: $value'));

// Collection literals with control-flow
var list = [
  1, 2,
  if (nums.isNotEmpty) 3,
  for (var n in nums) n * 10,
];

// Spread operator
var combined = [...nums, ...[5, 6]];
```

9. Object-Oriented Programming

```
class Person {
  String name;
  int age;

  Person(this.name, this.age);          // constructor shorthand

  Person.guest() : name = 'Guest', age = 0; // named constructor

  void greet() => print('Hi, I am $name');
}
```

```
class Student extends Person {
    String university;

    Student(String name, int age, this.university) : super(name, age);

    @override
    void greet() => print('Hi, I study at $university');
}
```

Interfaces & abstract classes:

```
abstract class Shape {
    double area();
}

class Circle implements Shape {
    double radius;
    Circle(this.radius);
    @override
    double area() => 3.14159 * radius * radius;
}
```

Mixins -- reusable behavior without inheritance:

```
mixin Logger {
    void log(String msg) => print('[LOG] $msg');
}

class Service with Logger {
    void run() => log('Service running');
}
```

Getters/setters:

```
class Rectangle {
    double width, height;
    Rectangle(this.width, this.height);
    double get area => width * height;
    set widthValue(double w) => width = w;
}
```

Extension methods -- add functionality to existing types:

```
extension StringExtension on String {
    String capitalize() => '${this[0].toUpperCase()}${substring(1)}';
}

print('dart'.capitalize()); // "Dart"
```

Extension types (Dart 3+) -- zero-cost wrapper types for type safety without runtime overhead:

```
extension type UserId(int id) {  
  bool get isValid => id > 0;  
}
```

10. Modern Dart 3.x Features

Records -- lightweight, anonymous, immutable value types (no class needed):

```
(String, int) getUser() => ('Ali', 21);  
var (name, age) = getUser();  
  
// Named fields  
({String name, int age}) user = (name: 'Ali', age: 21);  
print(user.name);
```

Pattern matching & destructuring:

```
var point = (3, 4);  
switch (point) {  
  case (0, 0):  
    print('Origin');  
  case (var x, var y) when x == y:  
    print('On diagonal');  
  default:  
    print('Point: $point');  
}  
  
// Destructuring in variable declarations  
var [first, second, ...rest] = [1, 2, 3, 4, 5];  
var {'name': userName, 'age': userAge} = {'name': 'Ali', 'age': 21};
```

Sealed classes -- exhaustive type hierarchies, great for state modeling:

```
sealed class Result<T> {}  
class Success<T> extends Result<T> { final T data; Success(this.data); }  
class Failure<T> extends Result<T> { final String error; Failure(this.error); }  
  
String handle(Result r) => switch (r) {  
  Success(data: var d) => 'Got: $d',  
  Failure(error: var e) => 'Error: $e',  
};
```

11. Primary Constructors (New in Dart 3.13)

Stable as of the August 2026 release, primary constructors let you declare a constructor and its instance variables directly in the class header, cutting boilerplate significantly.

```
// Old style
class Point {
  final double x, y;
  Point(this.x, this.y);
}

// New primary constructor style (Dart 3.13+)
class Point(double x, double y);
```

Dart 3.12 also introduced **private named parameters**, letting constructors initialize private fields directly through named initializing formals -- useful for cleaner encapsulated classes without extra boilerplate getters.

12. Asynchronous Programming

```
Future<String> fetchData() async {
  await Future.delayed(Duration(seconds: 2));
  return 'Data loaded';
}

void main() async {
  print('Fetching...');
  String data = await fetchData();
  print(data);
}

// Handling multiple futures
Future<void> loadAll() async {
  var results = await Future.wait([fetchData(), fetchData()]);
}

// Streams -- sequences of async events
Stream<int> countStream() async* {
  for (int i = 1; i <= 5; i++) {
    await Future.delayed(Duration(seconds: 1));
    yield i;
  }
}

void listenToStream() async {
  await for (var value in countStream()) {
    print(value);
  }
}
```


Isolates -- Dart's concurrency model. Instead of shared-memory threads, isolates run independently with their own memory and communicate via message passing -- this avoids race

conditions entirely and is ideal for CPU-heavy tasks (e.g. running an ML preprocessing step without blocking the UI thread in a Flutter app).

```
import 'dart:isolate';

void heavyTask(SendPort sendPort) {
  int result = 42; // some heavy computation
  sendPort.send(result);
}
```

13. Error Handling

```
try {
  int result = 10 ~/ 0;
} on IntegerDivisionByZeroException {
  print('Cannot divide by zero');
} on FormatException catch (e) {
  print('Format error: $e');
} catch (e, stackTrace) {
  print('Unknown error: $e');
} finally {
  print('Cleanup runs regardless');
}

// Custom exceptions
class InsufficientFundsException implements Exception {
  final String message;
  InsufficientFundsException(this.message);
  @override
  String toString() => 'InsufficientFundsException: $message';
}

void withdraw(double amount, double balance) {
  if (amount > balance) {
    throw InsufficientFundsException('Not enough balance');
  }
}
```

14. Packages, pubspec.yaml & Compilation Targets

Every Dart project has a pubspec.yaml defining dependencies and metadata:

```
name: my_app
description: A sample Dart project
version: 1.0.0
environment:
```

```

sdk: '^3.13.0'
dependencies:
  http: ^1.2.0
dev_dependencies:
  test: ^1.25.0

```

- Install packages: `dart pub get`
- Add a package: `dart pub add http`
- Browse packages at **pub.dev**

Compilation targets:

Target	Use case
JIT (Just-In-Time)	Development -- hot reload, fast iteration
AOT (Ahead-Of-Time)	Production native binaries -- instant startup
dart2js	Compile to JavaScript for web deployment
dart2wasm	Compile to WebAssembly for near-native web performance

15. Practice Exercises

1. Write a function using named + optional positional parameters that formats a user profile string.
2. Model a Shape hierarchy using sealed class and pattern-match over Circle, Square, Triangle to compute area.
3. Build a Stream-based countdown timer that prints a message every second and completes with "Done!".
4. Refactor a class with a traditional constructor into the new Dart 3.13 primary constructor syntax.
5. Write a custom exception for a login system and handle it with try/on/catch/finally.
6. Use records and destructuring to return multiple values from a function that calculates min/max/average of a list.

Quick Reference: Null-Safety Cheat Sheet

Operator	Meaning
?	Marks a type as nullable
??	Returns left value unless null, then right
??=	Assigns right value only if left is currently null
?.	Safe access -- returns null if object is null
!	Force non-null assertion (throws if actually null)

Operator	Meaning
late	Declares non-nullable var initialized later